

CHAPTER 1

INTRODUCTION

1.1 Landslide Monitoring and Early Warning Systems

A devastating natural disaster, landslides endanger people, buildings, and ecosystems all throughout the world. Climate change, urbanization, and deforestation are only a few of the variables that are increasing their devastating potential. To combat this, early warning systems and good monitoring are very necessary. Coverage, precision, and timeliness are three areas where traditional landslip monitoring techniques fall short, calling for new approaches that make use of cutting-edge technology. Modern advancements in artificial intelligence (AI), wireless sensor networks (WSNs), and the Internet of Things (IoT) have created exciting new opportunities for landslip early warning and monitoring systems. It is now possible to gather real-time data from hard-to-reach places, analyse it intelligently, and provide early warnings to lessen the impact of possible catastrophes by combining Internet of Things (IoT) devices with WSNs enabled by artificial intelligence.

In order to better monitor landslides and provide early warning systems, this research presents a thorough architecture for combining the Internet of Things (IoT) with WSNs that are enabled by artificial intelligence (AI). There are a number of benefits to using these technologies in tandem as opposed to more conventional methods. To start, a wide range of environmental variables, including soil moisture, rainfall, seismic activity, and slope stability, can be easily measured with the use of wireless sensors that have Internet of Things (IoT) capabilities. To provide thorough

coverage, these sensors can be strategically positioned in locations prone to landslides, such as inaccessible or rough terrain. Second, using AI algorithms makes it easier to intelligently analyse and analyse the data that has been acquired. It is possible to train machine learning algorithms to spot trends, outliers, and warning signs of upcoming landslides. These AI algorithms can learn from past data and input from sensor networks to become better predictors over time, making early warning systems more accurate and reliable. To add to that, stakeholders can react quickly to changing landslip hazards thanks to real-time monitoring and decision-making made possible by AI and the Internet of Things. The efficient transfer of data between sensor nodes, data processing centres, and end-users is made possible by modern communication protocols and cloud-based platforms. This allows for the quick distribution of warnings and advisories.

1.2 Objective

This study discusses the urgent need for early warning systems and monitoring systems for landslides, especially in areas vulnerable to such natural catastrophes. The suggested approach gets over problems like poor infrastructure and communication breakdowns by combining Wireless Sensor Networks (WSNs) with IoT devices and AI methods. It uses a number of sensors to measure things like soil moisture, rainfall, vibration, and humidity, among other things. Timely alerts and mitigation methods are made possible by processing this data using AI algorithms that identify prospective landslip occurrences in real-time. One potential way to make communities more resistant to landslip risks is to use the novel strategy outlined in this research.

CHAPTER 2

LITRETURE REVIEW

Bressani, L. et al. the management and safety of geotechnical operations, as well as the prevention and mitigation of landslip incidents, rely heavily on monitoring systems. New opportunities and difficulties have arisen for geotechnical engineers as a result of the convergence of inexpensive sensors with powerful data processing capabilities. There was a difficulty that has to be addressed: how to efficiently comprehend enormous data sets generated by Big Data. The potential quality of a vast amount of data should not be disregarded, even if it was not synonymous with an efficient system. As a result, improving the data quality obtained from big data systems necessitates the creation of new methods. One such method that, when designed and implemented properly, can synthesize massive amounts of data into crucial information for right decision making was artificial intelligence.

Harsa et al. these authors' research shows that ML and AI performed better once input data was appropriately pre-processed. During the model development phase, the author distinguish between two approaches to ML and AI model creation. In contrast to the second one, the first one was pre-processed. In addition, the second strategy achieved better results than the first, and it did so with fewer explanatory factors. In order to prepare the data for the second technique, the author used PCA and log-transformation as pre-processing steps. The first method's optimal model was given by the GLM, whereas the second one was given by the DL. Using rainfall

characteristics retrieved from satellites, the models predicted the likelihood of landslides occurring one day in the future.

Khoa, V. V., & Takayama, S. An event-driven system, the Landslip Monitoring System aims to lessen the likelihood of landslip damage. The system's measurement components were a wireless sensor network, making it a telemetric system. Sensors allow for the analysis of field state, network status, and individual nodes in both normal and emergency scenarios. Despite the system's limitations in catastrophe prediction, landslip monitoring in real-time enables further data analysis.

Ofli, F. et al. An interdisciplinary team consisting of computer scientists from the Qatar Computing Research Institute (QCRI), earthquake experts from the European-Mediterranean Seismological Centre (EMSC), and landslip experts from the British Geological Survey (BGS) worked together to develop the system that was showcased in this paper. Using cutting-edge AI methods, the created system automatically identifies information relevant to landslides by using real-time data from online social media.

Shutimarrungson, N., & Wuttidittachotti, P. these authors research proposes the creation of propagation models for landslip management systems that use wireless sensor networks. The measurement data and preexisting propagation models were used to build the models. The development's measuring setup included two primary cases: one with ground-based antennas and one with antennae placed one metre above ground. Both the first scenario's propagation model and the second scenario's multi-ray tracing model were built using these authors experimental data in conjunction with the log-normal model and the log-normal model, respectively.

Sruthy et al. In the realm of geophysical research, one of the current, demanding study fields was the real-time monitoring of landslides. The goal of an Internet of Things (IoT) landslip detection system was to identify potential danger signs, alert authorities in a timely manner, and ultimately prevent casualties. The existing methods for detecting landslides were not as precise.

Valade et al. 14 Using data downloaded and processed automatically from Sentinel-1 SAR, Sentinel-2 SWIR, and Sentinel-5P TROPOMI, the author provide a functioning volcano monitoring system. The data that has been retrieved was intended to provide important metrics that can provide information on the level of volcanic activity. These parameters include changes in surface reflectance, surface heat anomalies, surface deformation, and SO₂ gas emissions.

CHPATER 3

SYSTEM SPECIFICATION

3.1 System Requirements

System Component	Specifications
PROCESSOR	Intel(R) Pentium(R) CPU G2030 @ 3.00GHz, 3.00 GHz or Intel i3.
STORAGE	ABOVE 2GB
RAM	Minimum of 4GB. The recommended amount can be 6GB.
HARD DISK	3GB of available hard-disk space for installation, additional free space is required for work space.

Table 3.1 System Requirements

3.2 Hardware Requirement

S. No.	Hardware Component	Quantity
1.	Arduino	1
2.	Node MCU	1
3.	Soil Moisture Sensor	1
4.	Rainfall Sensor	1
5.	Vibration Sensor	1
6.	Humidity Sensor	1
7.	Buzzer	1
8.	Power Supply Module	1
9.	GSM Module	1
10.	Sim Card	1
11.	Connection Wires	20

Table 3.2 Hardware Requirements

3.3 Software Requirements

3.3.1 Arduino IDE

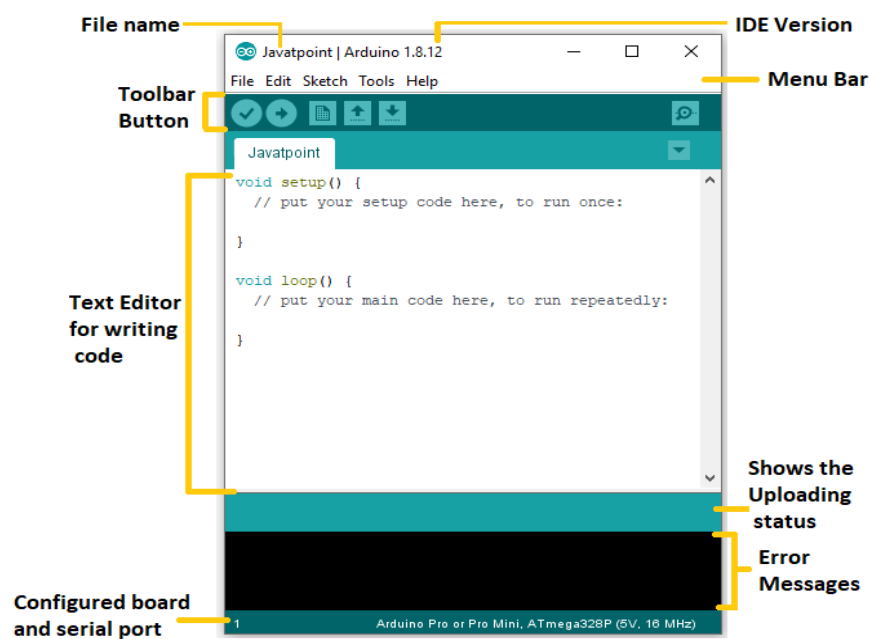


Figure 3.1 Arduino IDE Interface

The Arduino IDE is an open-source software, which is used to write and upload code to the Arduino boards. The IDE application is suitable for different operating systems such as Windows, Mac OS X, and Linux. It supports the programming languages C and C++. Here, IDE stands for Integrated Development Environment. The program or code written in the Arduino IDE is often called as sketching. We need to connect the Genuino and Arduino board with the IDE to upload the sketch written in the Arduino IDE software. The sketch is saved with the extension '.ino.'

Toolbar Button

The icons displayed on the toolbar are **New**, **Open**, **Save**, **Upload**, and **Verify**.

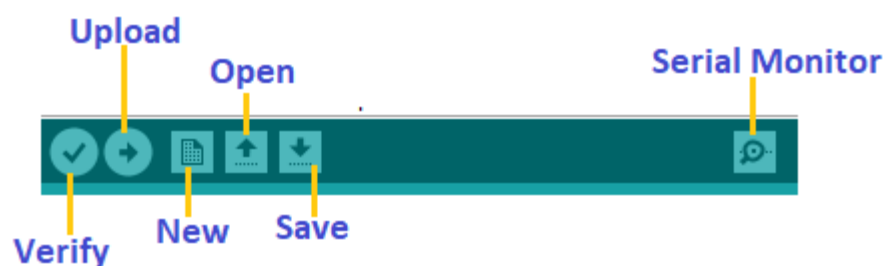


Figure 3.2 Arduino IDE Toolbar

3.3.2 Visual Studio

Visual Studio is an Integrated Development Environment (IDE) developed by Microsoft to develop Desktop applications, GUI (Graphical User Interface), console, web applications, mobile applications, cloud, and web services, etc. With the help of this IDE, you can create managed code as well as native code. It uses the various platforms of Microsoft software development software like Windows store, Microsoft Silverlight, and Windows API, etc. It is not a language-specific IDE as you can use this to write code in C#, C++, VB (Visual Basic), Python, JavaScript, and many more languages. It provides support for 36 different programming languages. It is available for Windows as well as for macOS.

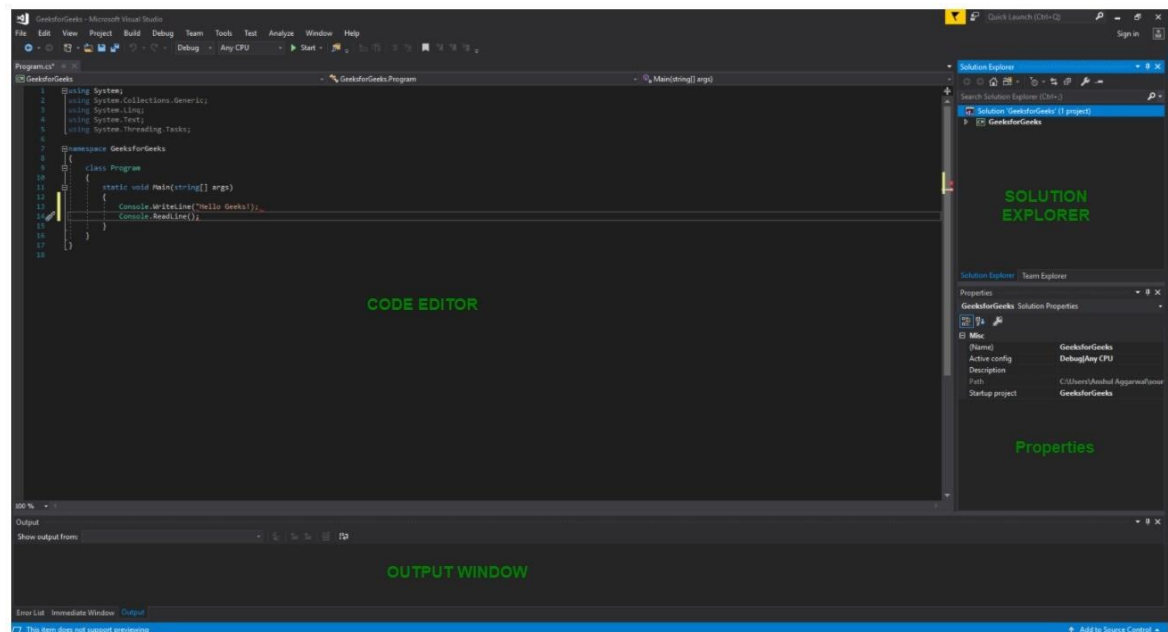


Figure 3.3 Visual Studio Interface

Visual Studio Editions

There are 3 editions of Microsoft Visual Studio as follows:

1. Community

It is a free version which is announced in 2014. *All other editions are paid.* This contains the features similar to Professional edition. Using this edition, any individual developer can develop their own free or paid apps like *.Net applications*, Web applications and many more. In an enterprise organization, this edition has some limitations. For example, if your organization have more than 250 PCs and having annual revenue greater than \$1 Million (US Dollars) then you are not permitted to use this edition. In a non-enterprise organization, up to five users can

use this edition. Its main purpose is to provide the Ecosystem (Access to thousands of extensions) and Languages (You can code in C#, VB, F#, C++, HTML, JavaScript, Python, etc.) support.

2. Professional

It is the commercial edition of Visual Studio. It comes in Visual Studio 2010 and later versions. It provides the support for XML and XSLT editing and includes the tool like Server Explorer and integration with Microsoft SQL Server. Microsoft provides a free trial of this edition and after the trial period, the user has to pay to continue using it. Its main purpose is to provide Flexibility (Professional developer tools for building any application type), Productivity (Powerful features such as CodeLens improve your team's productivity), Collaboration (Agile project planning tools, charts, etc.) and Subscriber benefits like Microsoft software, plus Azure, Pluralsight, etc.

3. Enterprise

It is an integrated, end to end solution for teams of any size with the demanding quality and scale needs. Microsoft provides a 90-days free trial of this edition and after the trial period, the user has to pay to continue using it. The main benefit of this edition is that it is highly scalable and deliver high-quality software.

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 Block Structure

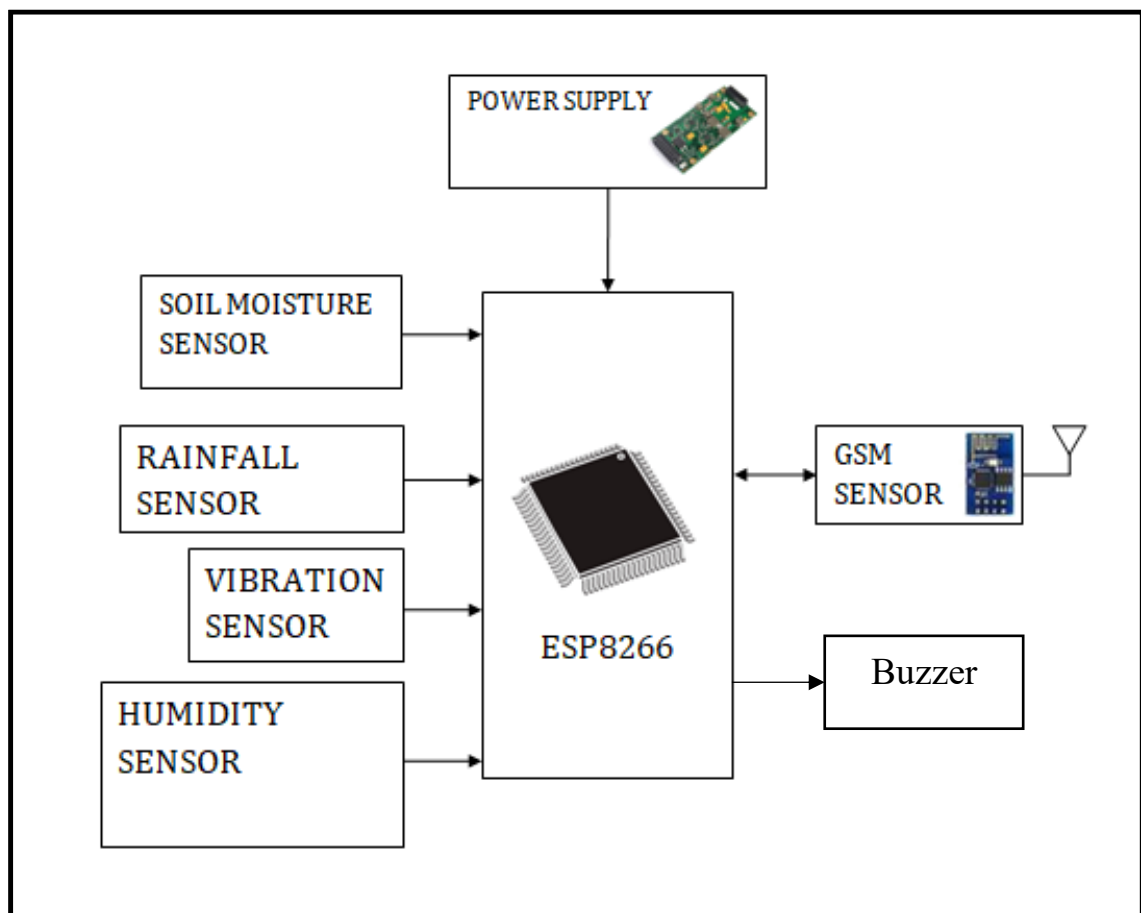


Figure 4.1 Block Diagram

4.1.1 Node MCU Esp8266

NodeMCU Esp8266 is a low-cost open source IoT platform. Its original hardware components were ESP-12 modules and firmware based on the ESP8266

Wi-Fi SoC. The NodeMCU Dev Kit/Board includes the ESP8266 Wi-Fi chip. The TCP/IP protocol is used by Espressif Systems' low-cost ESP8266 Wi-Fi chip. To learn more about the ESP8266, use the Wi-Fi Module. The NodeMCU microcontroller and development board are designed primarily for Internet of Things-related purposes

4.1.2 Soil Moisture Sensor

Soil moisture sensors are devices designed to measure the water content in soil, an important parameter for understanding the hydration level and guiding irrigation in agriculture, horticulture, and environmental science. These sensors help optimize water usage, improve crop yield, and minimize environmental impact by preventing over- or under-watering. Soil moisture sensors are devices designed to measure the water content in soil, an important parameter for understanding the hydration level and guiding irrigation in agriculture, horticulture, and environmental science. These sensors help optimize water usage, improve crop yield, and minimize environmental impact by preventing over- or under-watering. Soil moisture sensors are devices designed to measure the water content in soil, an important parameter for understanding the hydration level and guiding irrigation in agriculture, horticulture, and environmental science. These sensors help optimize water usage, improve crop yield, and minimize environmental impact by preventing over- or under-watering. A single sensor provides moisture data for a limited volume of soil, necessitating multiple sensors to accurately assess larger areas due to soil heterogeneity. High-accuracy sensors like TDR and neutron moisture gauges can be expensive, limiting their accessibility for small-scale applications.



Figure 4.2 Soil Sensor

4.1.3 Rainfall Sensor

Rainfall sensors generally operate based on the detection of water on their sensing surface. The presence of water alters the sensor's physical or electrical properties, triggering a response in the connected system. The exact mechanism of detection can vary depending on the type of rain sensor. Rainfall sensor, also known as a rain sensor, is a device used to detect rain and provide signals that can trigger a response in other systems, such as automatic irrigation systems, weather stations, and automotive windshield wipers. The primary purpose of a rainfall sensor is to act as a switch that activates or deactivates mechanisms based on the presence of precipitation.



Figure 4.3 Rainfall Sensor

4.1.4. GSM

The primary global mobile standard and the communications technology with the highest rate of growth is GSM. From the perspective of the mobile operator, a GSM modem is similar to a cell phone. A computer may use the mobile network for communication when a GSM modem is attached. Although many of these GSM modems can also send and receive SMS and MMS messages, they are frequently used for mobile internet access. A mobile phone with GSM modem functionality can also function as a standalone modem via USB, Bluetooth, or serial connection.



Figure 4.4 GSM Module

4.1.6 12V Power Supplies

The 12V (or 12VDC) supply is one of the most widely utilized power sources available today. To convert a 120VAC or 240VAC input into a 12VDC output, a transformer, diode, and transistor combination is frequently used. The two different forms of 12V power sources are regulated and uncontrolled power supplies. Furthermore, in an acopian switching regulated power supply, substantial EMI filtering and shielding are employed to minimize noise transmitted to the line and load in both common and differential modes. The system's power source, represented by the power supply block in the diagram, steps down the voltage from 230V to 5V in order to provide the system with the vital power it needs. The voltage is then further passed to the remaining system blocks after being stepped down to 5V.

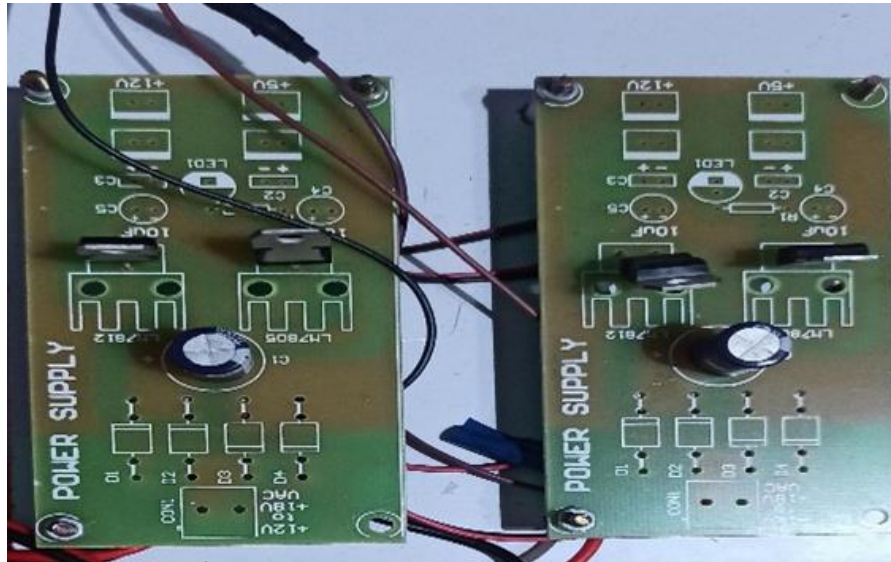


Figure 4.6 Power Supply Module

4.1.7 Vibration Sensor

Vibration sensors work by converting mechanical vibration into an electrical signal. The specific principle of operation can vary depending on the type of sensor. Common mechanisms involve piezoelectric elements, which generate a voltage when mechanical force is applied, or capacitive components, where vibration alters the capacitance between sensor elements, which can be measured and translated into vibration data.

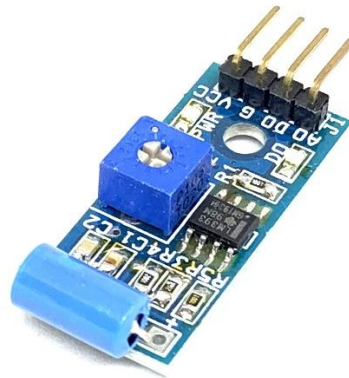


Figure 4.7 Vibration Sensor

4.1.7 Buzzer

A buzzer is an efficient component to include the features of sound in our system or project. It is an extremely small & solid two-pin device thus it can be simply utilized on breadboard or PCB. So in most applications, this component is widely used. There are two kinds of buzzers commonly available like simple and readymade. Once a simple type is power-driven then it will generate a beep sound continuously. A readymade type looks heavier & generates a Beep. Beep. Beep. This sound is because of the internal oscillating circuit within it. This buzzer uses a DC power supply that ranges from 4V – 9V. To operate this, a 9V battery is used but it is suggested to utilize a regulated +5V/+6V DC supply. Generally, it is connected through a switching circuit to switch ON/OFF the buzzer at the necessary time interval.



Figure 4.8 Buzzer

4.1.8 Humidity Sensor

Humidity sensors measure the amount of water vapor in the air. The principle of operation varies among different types of sensors, but they typically measure either relative humidity (RH), which is the amount of moisture in the air compared to the maximum moisture the air can hold at a given temperature, or absolute humidity, which is the total moisture in the air regardless of temperature. Sensors convert these measurements into electrical signals that can be easily read and interpreted.

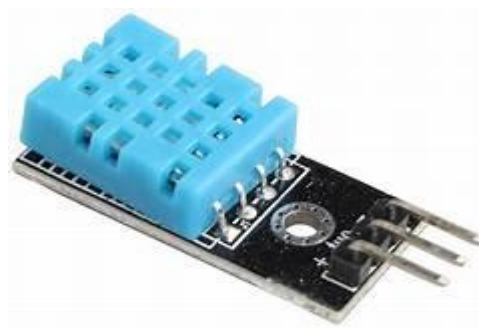


Figure 4.8 Humidity Sensor

CHAPTER 5

IMPLEMENTATION

5.1 Working

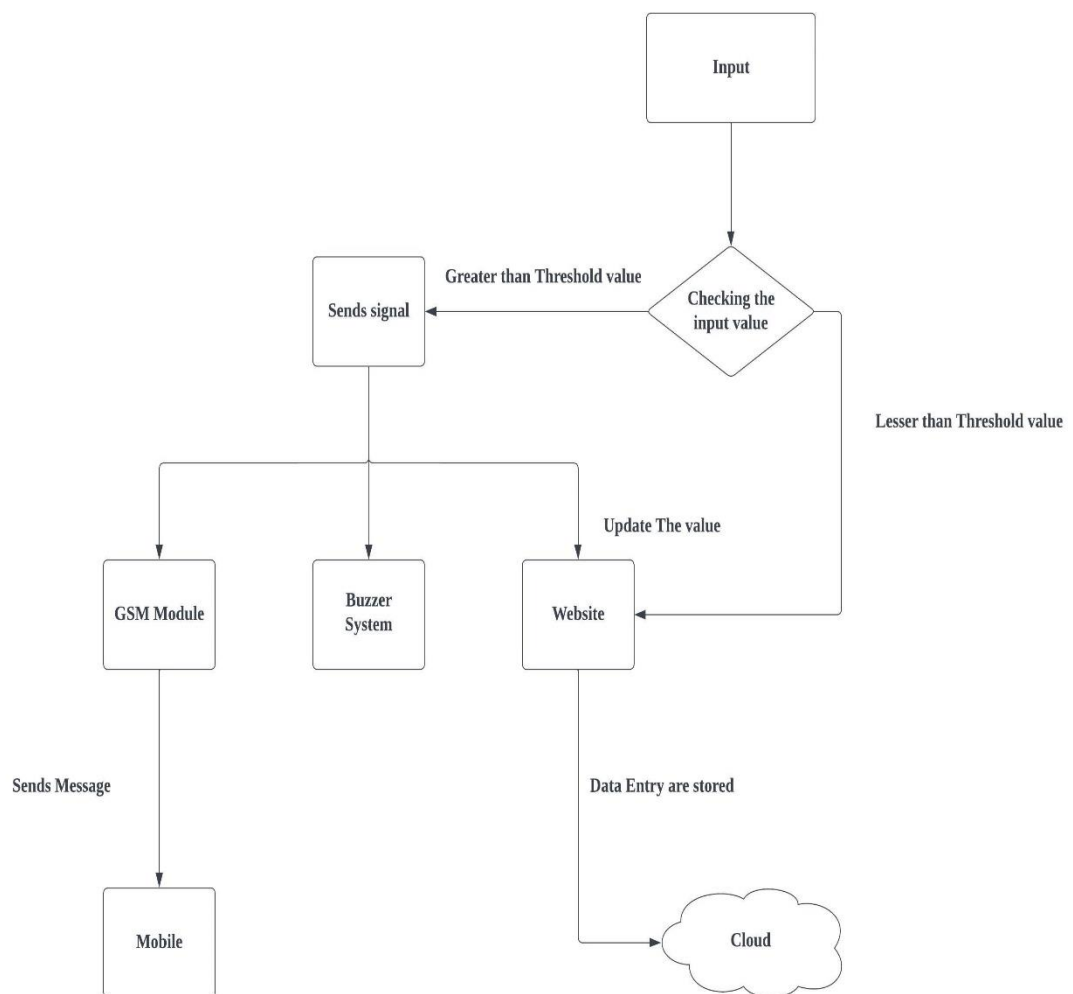


Figure 5.1 Work Flow Diagram

Process

Data Collection: Sensors deployed in landslide-prone areas continuously monitor various parameters such as ground movement, slope inclination, soil moisture, and rainfall. These sensors can include accelerometers, inclinometers, GPS devices, weather stations, and more. Data from these sensors are collected at regular intervals.

Data Processing and Analysis: The collected sensor data are transmitted to a central processing unit, which may be located on-site or remotely. Here, the data undergo pre-processing to remove noise and anomalies. Sophisticated algorithms are then applied to analyse the data for patterns indicative of potential landslide events. These algorithms may utilize machine learning, statistical analysis, or other techniques to identify trends and anomalies in the sensor data.

Landslide Detection: Once the data analysis identifies abnormal patterns or deviations from expected behaviour, it triggers the detection of a potential landslide event. This detection is based on predefined thresholds or criteria set by experts to differentiate between normal and abnormal conditions. The system may also consider historical data and predictive models to enhance the accuracy of detection.

Alert Generation: Upon detecting a potential landslide event, the system generates alerts to notify relevant stakeholders. This alert can be in the form of visual notifications on monitoring dashboards, as well as audible alerts through sirens or automated phone calls. Additionally, alerts may be sent via SMS, email, or other

communication channels to emergency response teams, local authorities, and residents in the affected areas.

Response and Mitigation: Upon receiving the alert, stakeholders can take appropriate actions to mitigate the risks associated with the potential landslide. This may include evacuating residents, closing roads or infrastructure, reinforcing slopes, or implementing other preventive measures to minimize the impact of the landslide event.

Monitoring and Evaluation: The system continues to monitor the situation in real-time, providing ongoing updates on the status of the landslide and any changes in conditions. After the event, post-incident analysis is conducted to evaluate the effectiveness of the detection system and identify areas for improvement.

5.2 Source Code

5.2.1 Controller code

```
#include <Wire.h>
#include <ESP8266WiFi.h>
#include <SoftwareSerial.h>
#include "mux_read.h"
#include "dht.h"

const char* host = "insourcetechnologies.org";
unsigned long last_data_upload_time;
```

```

int vibration, humidity, rain, moisture, vibration_data_uploaded,
moisture_data_uploaded, rain_data_uploaded, event_occured;

int vibe_time;

int MUX_SELECT_A = D1;

int VIBRATION_SENSOR = D0;

int BUZZER = D7;

String prediction;

String location = "10.9275873,76.9250553";

unsigned long lastExecutionTime, vibrationCheckTime;

dht DHT;

#define DHT11_PIN D5

void NeoGSMInit()
{
    Serial.print("AT");
    Serial.write('\r');
    Serial.write('\n');
    delay(300);
    Serial.print("AT+CREG=1");
    Serial.write('\r');
    Serial.write('\n');
    delay(300);
    Serial.print("AT+CSMS=1");
    Serial.write('\r');
    Serial.write('\n');
    delay(300);

```

```
Serial.print("AT+CMGF=1");
```

```
Serial.write('\r');
```

```
Serial.write('\n');
```

```
delay(300);
```

```
Serial.print("AT+CNMI=2,2,0,0,0");
```

```
Serial.write('\r');
```

```
Serial.write('\n');
```

```
delay(300);
```

```
Serial.print("AT+CSCS=\"GSM\"");
```

```
Serial.write('\r');
```

```
Serial.write('\n');
```

```
delay(300);
```

```
Serial.print("ATE1");
```

```
Serial.write('\r');
```

```
Serial.write('\n');
```

```
delay(300);
```

```
}
```

```
void SendMsg(String number, String msg)
```

```
{
```

```
Serial.print("AT+CMGS=\"");
```

```
Serial.print(number);
```

```
Serial.write("");
```

```
Serial.write('\r');  
    delay(200);  
    Serial.print(msg);  
    Serial.write(26); //Send CTRL+Z  
    delay(2000);  
}
```

```
void setup()  
{  
    Serial.begin(9600);  
    pinMode(VIBRATION_SENSOR, INPUT);  
    pinMode(BUZZER, OUTPUT);  
  
    muxInit(1, MUX_SELECT_A, NO_PIN, NO_PIN);  
    NeoGSMInit();  
    initWiFi("PROJECT", "withlove", 1);  
}
```

```
void loop()  
{  
  
    rain = (1023 - muxRead(0)) / 10;  
    delay(300);  
}
```

```
moisture = (1023 - muxRead(1)) / 10;
    delay(300);

int chk = DHT.read11(DHT11_PIN);

switch (chk)
{
    case DHTLIB_OK:
        //Serial.print("Humidity OK,\t");

break;

    case DHTLIB_ERROR_CHECKSUM:
        //Serial.print("Humidity Checksum error,\t");
        break;
    case DHTLIB_ERROR_TIMEOUT:
        // Serial.print("Humidity Time out error,\t");
        break;
    default:
        // Serial.print("Humidity Unknown error,\t");
        break;
}

if(DHT.humidity != DHTLIB_INVALID_VALUE)
```

```

{
    humidity = (int)DHT.humidity;
    // Serial.print("\tHM: ");
    // Serial.print(humidity);
    // Serial.println("");
}

    if(digitalRead(VIBRATION_SENSOR) == HIGH && vibration == 0)
    {
        delay(300);
        vibration = 1;
        vibe_time = millis();
        digitalWrite(BUZZER,HIGH);
        delay(2000);
        digitalWrite(BUZZER,LOW);
        SendMsg("+919994194928", "Earthquake landslide");
        delay(3000);

        // Serial.println("vibration");
        prediction = "Earthquake_landslide";
        String url = "/my-
        projects/her23119_iot_landslide/addEvent.php?event_occured=1&rain=";
        url += String(rain);
        url += "&moisture=";
    }

```

```
url += moisture;
    url += "&humidity=";
    url += humidity;
    url += "&vibration=";
    url += vibration;
    url += "&prediction=";
    url += prediction;
    url += "&location=";
    url += location;

    requestURL(host, url);

}
else if(digitalRead(VIBRATION_SENSOR) == 0)
{
    vibration = 0;
    vibe_time = millis();
    digitalWrite(BUZZER,LOW);
}

if(moisture > 50 && moisture_data_uploaded == 0)
{
```

```

moisture_data_uploaded = 1;
    prediction = "Landslide";
    // Serial.println("moisture");

    String url = "/my-
projects/her23119_iot_landslide/addEvent.php?event_occured=1&rain=";
    url += String(rain);
    url += "&moisture=";
    url += moisture;
    url += "&humidity=";
    url += humidity;
    url += "&vibration=";
    url += vibration;
    url += "&prediction=";
    url += prediction;
    url += "&location=";
    url += location;
    requestURL(host, url);
    digitalWrite(BUZZER,HIGH);
    delay(2000);
    digitalWrite(BUZZER,LOW);
    SendMsg("+919994194928", "Landslide");

    delay(3000);
    delay(100);
}
else if(moisture < 50)

```

```

{
    moisture_data_uploaded = 0;
    digitalWrite(BUZZER,LOW);
}

if(rain >40 && rain_data_uploaded == 0 && moisture > 50)
{
    rain_data_uploaded = 1;

    // Serial.println("rain");
    prediction = "Rainfall_landslide";

    String url = "/my-
projects/her23119_iot_landslide/addEvent.php?event_occured=1&rain=";
    url += String(rain);
    url += "&moisture=";
    url += moisture;
    url += "&humidity=";
    url += humidity;
    url += "&vibration=";
    url += vibration;
    url += "&prediction=";
    url += prediction;
    url += "&location=";
    url += location;
    requestURL(host, url);

```

```

digitalWrite(BUZZER,HIGH);
    delay(2000);
    digitalWrite(BUZZER,LOW);
    SendMsg("+919994194928", "Rainfall landslide");

    delay(3000);

    delay(100);
}
else if(rain < 40)
{
    rain_data_uploaded = 0;

}

    if (millis() - lastExecutionTime >= 4000)
    {
        lastExecutionTime = millis();

        String url = "/my-
projects/her23119_iot_landslide/addEvent.php?rain=";
        url += String(rain);
        url += "&moisture=";
        url += moisture;
    }
}

```

```

url += "&humidity=";
url += humidity;
url += "&vibration=";
url += vibration;

    requestURL(host, url);

}

    delay(50);

}

```

WEBPAGE CODING:-

```

<?php
    include('session.php');
?>
<!DOCTYPE html>
<html >

<head>
    <meta charset="UTF-8">
    <title>IoT Landslide Detection</title>

    <link rel="stylesheet" href="css/reset.min.css">
    <link rel="stylesheet" href="css/style.php?theme=purple">

```

```

<!-------
//Auto Reload Page using AJAX
----->
<script src="js/jquery.min.js"></script>
<script type="text/javascript">
function delay(ms){
    var start = new Date().getTime();
    var end = start;
    while(end < start + ms) {
        end = new Date().getTime();
    }
}
$(document).ready(function()
{
    function functionToLoadFile()
    {
        jQuery.get('getVariables.php?keywords>refresh=1',
function(data)
        {
            if (data == "1")
            {
                document.getElementById('buzzer').play();
                delay(1000);
                $(location).attr('href', 'app.php');

                jQuery.get('setVariables.php?keywords>refresh=0');

```

```

    }

        setTimeout(functionToLoadFile, 5000);

    });

}

setTimeout(functionToLoadFile, 10);

});

</script>

<script>
function autoRefresh() {
    window.location = window.location.href;
}
setInterval('autoRefresh()', 5000);
</script>

<!-------
//Auto Reload Page using AJAX - END
----->

</head>

<body style="background-image: url(bg.jpg)">
    <div class="cta" style="float: right; padding-top: 7px;">

```

```

<span class="button-small">
Welcome <?= ucfirst($_SESSION['normal_user']). " "; ?><em><a
href="logout.php">Logout</a></em></span>

</div>

<div class="pen-title">
    <h1>IoT Landslide Detection</h1>
</div>

<audio id="buzzer"><source src="beep.ogg" type="audio/ogg"></audio>

<div class="form-module form-module-large">
    <ul class="top-menu">
        <li><a class="active" href="app.php">Home</a></li>
    </ul>

    <div class="content">
        <h2 class="headings">Current Parameters</h2>
        <?php

```

```

        $rain = get_default_value('rain');
        $moisture = get_default_value('moisture');
        $Humidity =
get_default_value('humidity');
        $vibration =
get_default_value('vibration');

        //$prediction = "Safe Condition";
        $moisture_alert = "";

```

```

$rain_alert = "";
$vibration_alert = "";

if($rain > 40 && $moisture > 45)
{
    $moisture_alert = "style='
background: red; color: white;";
    // $rain_alert = "style=' background:
red; color: white;";
    // $prediction = "Rain Fall
Landslide";

    $rain_alert = "red";
    $moisture_alert = "red";

}
else if($moisture > 50 && $moisture !=
"")
{
    $moisture_alert = "style='
background: red; color: white;";

    // $prediction = "Landslide";
    $moisture_alert = "red";

}
else if($rain > 40)
{
    $rain_alert = "red";

}

```

else

```
{
    $rain_alert = "green";
    $moisture_alert = "green";
}
```

```
if($vibration == 1 && $vibration != "")
{
```

```
    $vibration_alert = "style='
background: red; color: white;";

    //$prediction = "Earthquake
Landslide";

}
```

```
echo "<div style='float: left; margin: 0 30px
30px;'><h2>Rain</h2> <br/> <br/>";
```

```
echo "<a class='button-large $rain_alert'>" .
get_default_value('rain') . "</a></div>";
```

```
echo "<div style='float: left; margin: 0 30px
30px;'><h2>Moisture</h2> <br/> <br/>";
```

```
echo "<a class='button-large $moisture_alert'>" .
get_default_value('moisture') . "</a></div>";
```

```
//echo "<div style='clear: both;'> </div> <br/> <br/>";
```

```

        echo "<div style='float: left; margin: 0 30px
30px;'><h2>Humidity</h2> <br/> <br/>";

```

```

        echo "<a class='button-large'>" .
get_default_value('humidity') . "</a></div>";

```

```

?>

```

```

<div style="clear: both;"> </div>

```

```

<br/>

```

```

</div>

```

```

<div class="content">

```

```

    <h2 class="headings">Event History</h2>

```

```

    <div style="clear: both;"> </div>

```

```

    <table>

```

```

        <tr>

```

```

            <th>S. No.</th>

```

```

            <th>Rain</th>

```

```

            <th>Moisture</th>

```

```

            <th>Humidity</th>

```

```

            <th>Vibration</th>

```

```

            <th>Prediction</th>

```

```

            <th>location</th>

```

```

            <th>Date</th>

```

```

            <th>Time</th>

```

```

            <th>Clear</th>

```

```

        </tr>

```

```

<?php

function table_function($key)
{

    /*

        $rain = get_default_value('rain');
        $moisture = get_default_value('moisture');
        $humidity =
get_default_value('humidity');
        $vibration =
get_default_value('vibration');

        //$prediction = "Safe Condition";

        $moisture_alert = "";
        $rain_alert = "";
        $vibration_alert = "";

        if($rain > 30 && $moisture > 40)
        {
            //$moisture_alert = "style='
background: red; color: white;";
            //$rain_alert = "style=' background:
red; color: white;";
            //$prediction = "Rain Fall
Landslide";
        }

```

```

else if($moisture > 55 && $moisture != "")
    {
        // $rain_alert = "style=' background:
red; color: white;";
        // $prediction = "Landslide";
    }

    if($vibration == 1 && $vibration != "")
    {
        $vibration_alert = "style='
background: red; color: white;";
        // $prediction = "Earthquake
Landslide";
    }

    */
    echo "<td>$key</td>";
    echo "<td>" .
get_ini_value("rain_history", "$key") . "</td>";
    echo "<td>" .
get_ini_value("moisture_history", "$key") . "</td>";
    echo "<td>" .
get_ini_value("humidity_history", "$key") . "</td>";
    echo "<td>" .
get_ini_value("vibration_history", "$key") . "</td>";
    echo "<td>" .
get_ini_value("prediction_history", "$key") . "</td>";

```

```

echo "<td>" . get_ini_value("location", "$key") . "</td>";

                                //echo "<td>" . $prediction . "</td>";

                                echo "<td>" .
get_ini_value("date_history", "$key") . "</td>";

                                echo "<td>" .
get_ini_value("time_history", "$key") . "</td>";

                                echo "<td><a
href='deleteVariables.php?return_back=1&return_url=app.php&RECORD=$key'><
img src='remove.png'></a></td></tr>";

                                }

                                foreach_ini_value("table_function");

                                ?>

                                </table>

                                <div style="clear: both;"> </div>

                                <br/>

                                </div>

</body>

</html>

```

CHAPTER 6

SCREENSHOTS AND OUTPUTS

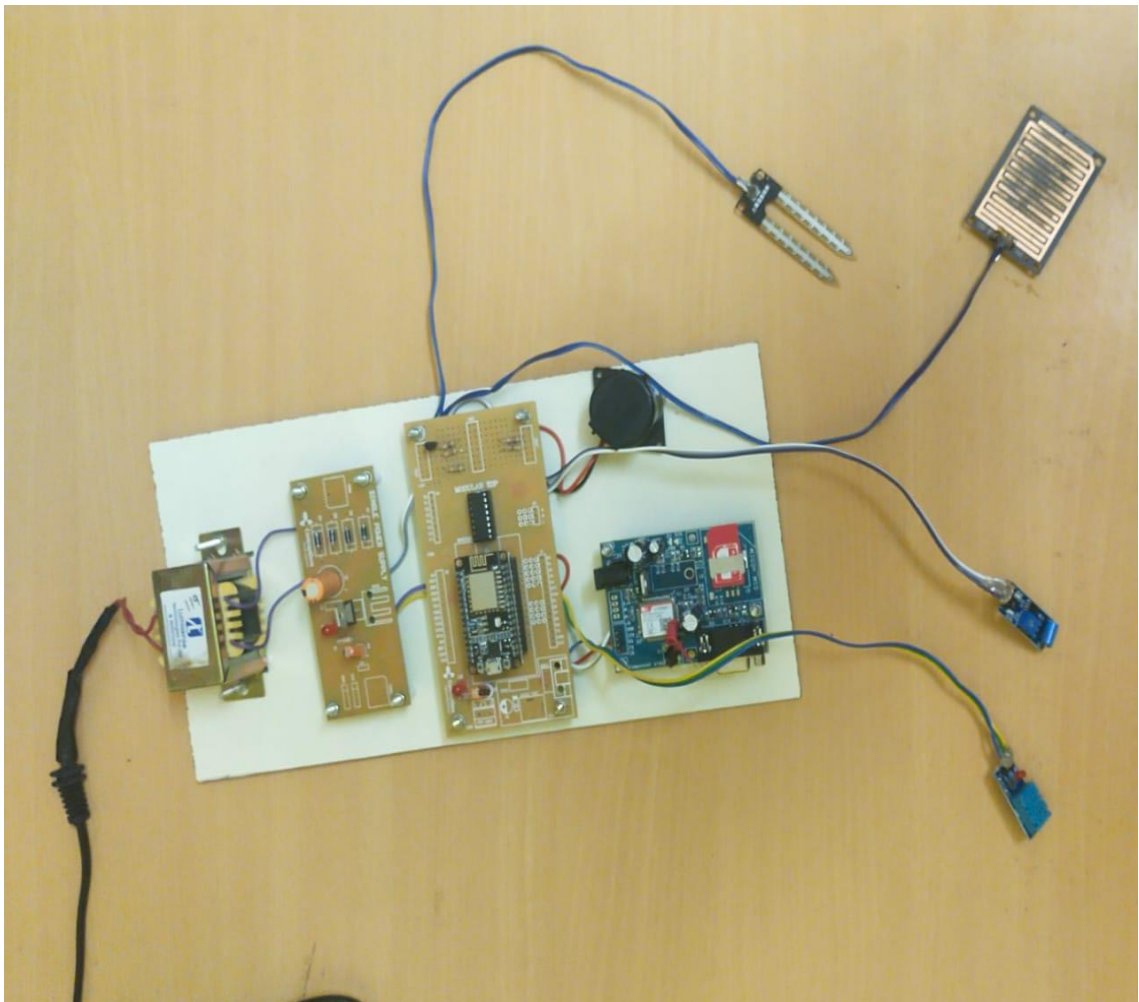


Figure 6.1 Circuit Board

Above Figure 6.1 shows the Unpowered State of the project circuit board.

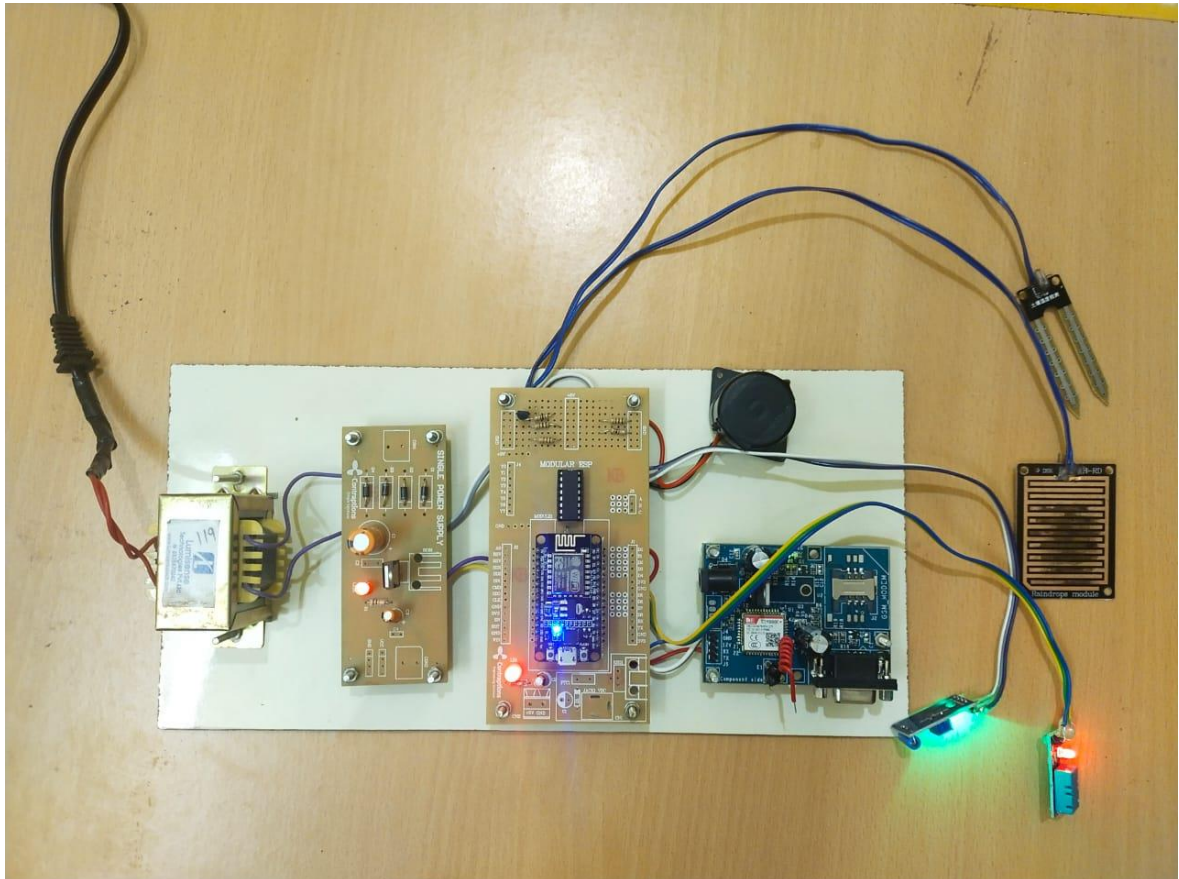


Figure 6.2 Powered Circuit Board

Above Figure 6.2 Shows the powered status of the project circuit Board. Which was in an Idle State.

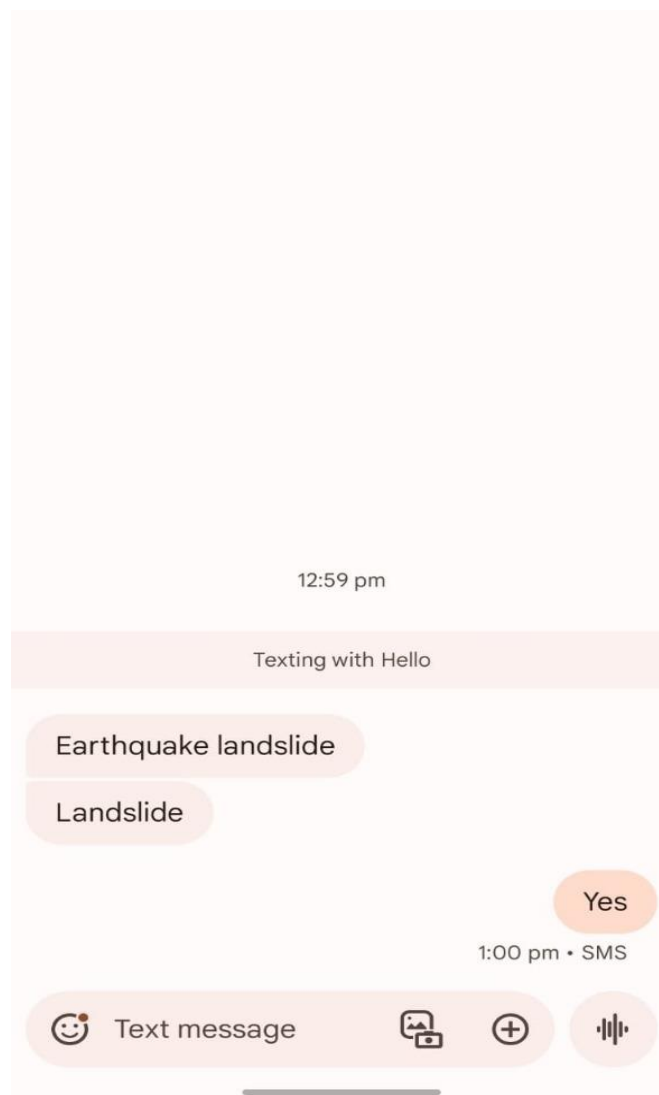


Figure 6.3 Message Alert

Above Figure 6.3 Message Alerts Shows the Image, that the information had deleivered to the peoples.

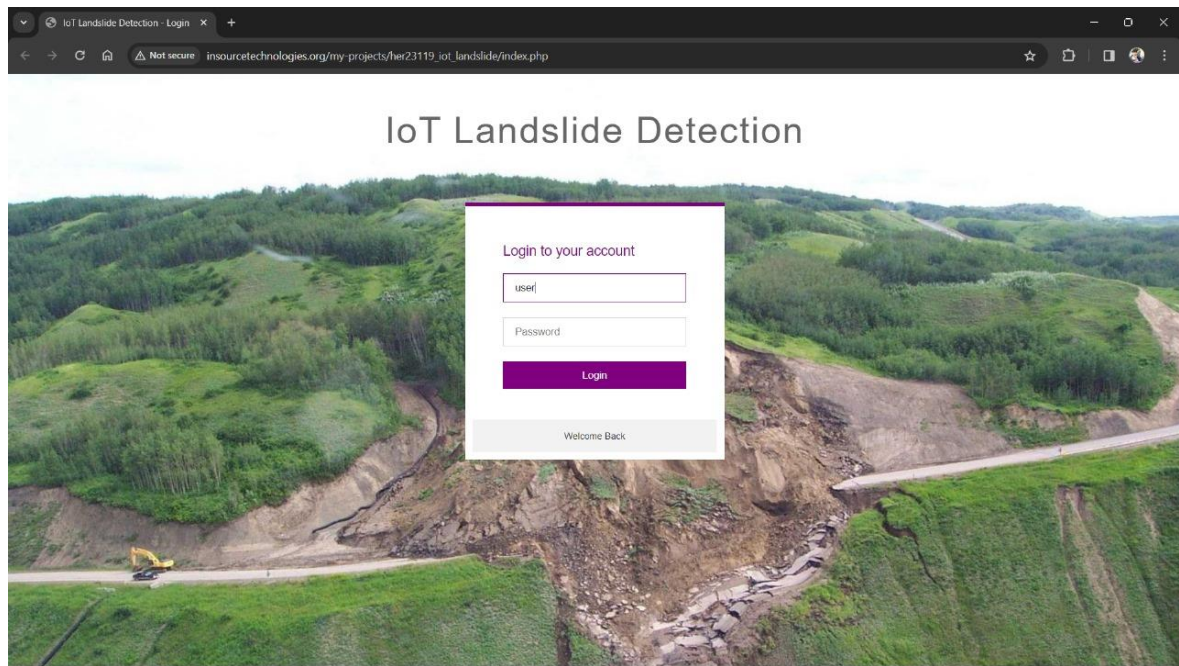


Figure 6.4 Login Page

Above Figure 6.4 shows the Login page of the Landslide Monitoring System, it requires the User ID and the Password for Access.

A close-up view of the login form from the previous image. It shows the title 'Login to your account' in purple. Below it are two white input fields with purple borders. The first field contains the text 'user|' and the second field contains the text 'Password'. A purple button with the word 'Login' in white is positioned below the fields. At the bottom of the form is a light gray bar with the text 'Welcome Back' in a small, dark font.

Figure 6.5 Pin Entry

Above Figure 6.5 shows the Pin Entry Page, which has a field to enter the ATM Pin and a Button to allow access to the ATM user.



Figure 6.6 Soil Moisture Sensing

Soil Moisture Sensor analyzing and gathering data from the sample will look as shown in the above Figure 6.6.

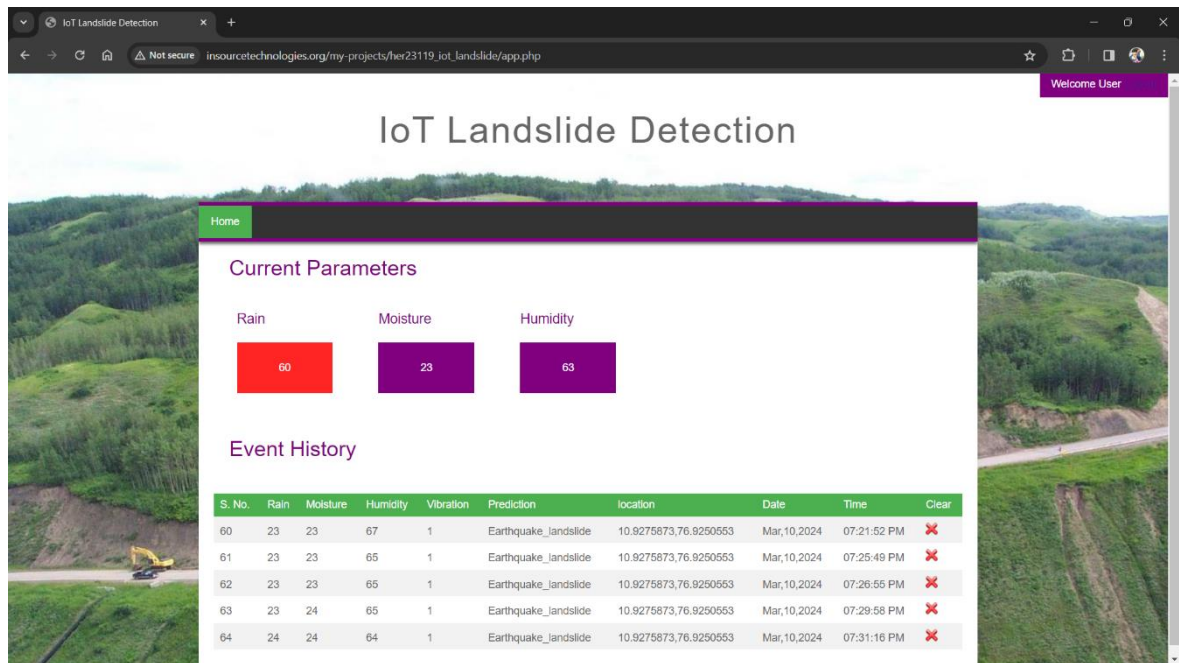


Figure 6.7 Soil Moisture Alert

Soil Moisture has been sensed and shown in the web page as shown in the above Figure 6.7.

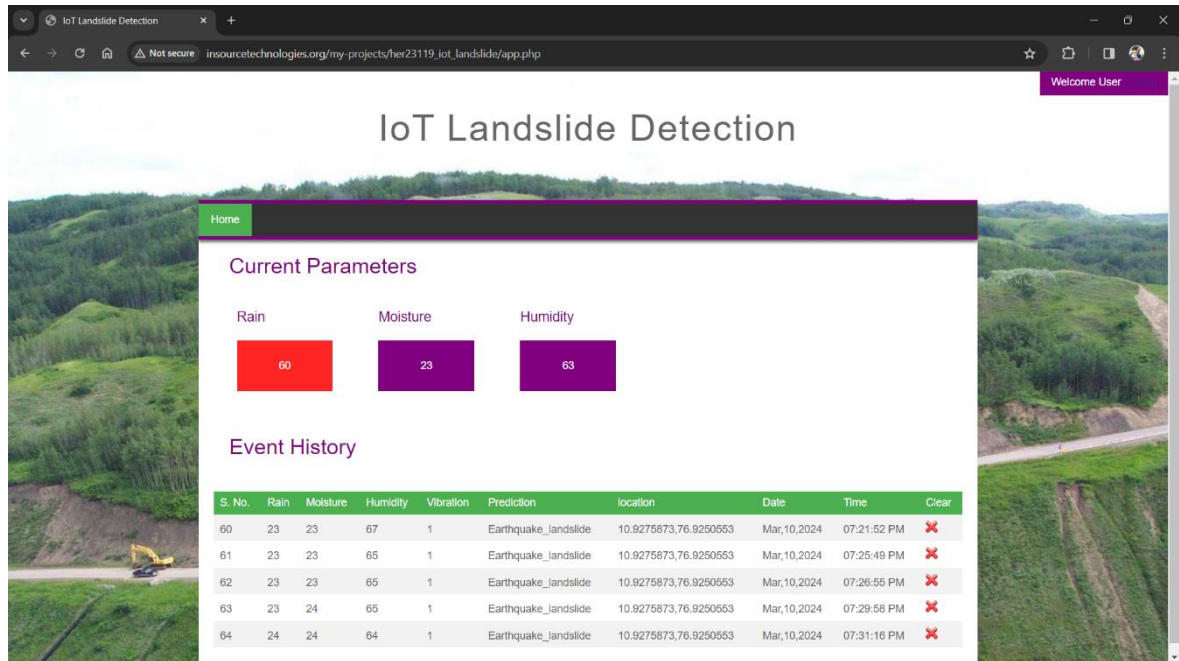


Figure 6.8 Rainfall Alert

Rainfall level has been sensed and shown in the web page as shown in the above Figure 6.8.

S. No.	Rain	Moisture	Humidity	Vibration	Prediction	location	Date	Time	Clear
60	23	23	67	1	Earthquake_landslide	10.9275873,76.9250553	Mar,10,2024	07:21:52 PM	✖
61	23	23	65	1	Earthquake_landslide	10.9275873,76.9250553	Mar,10,2024	07:25:49 PM	✖
62	23	23	65	1	Earthquake_landslide	10.9275873,76.9250553	Mar,10,2024	07:26:55 PM	✖
63	23	24	65	1	Earthquake_landslide	10.9275873,76.9250553	Mar,10,2024	07:29:58 PM	✖

Figure 6.9 Vibration Sensor Alert

Vibration has been sensed and danger shown in the web page as shown in the above Figure 6.7.

CHAPTER 7

CONCLUSION

To sum up, the suggested landslide monitoring system is a huge step forward in dealing with the problems of environmental monitoring, especially in areas vulnerable to landslides but without the necessary infrastructure. Integrating AI approaches, Internet of Things (IoT) devices, and Wireless Sensor Networks (WSNs), the system provides a complete and dependable solution for real-time landslip risk detection and assessment. Strong data collecting and transmission capabilities, even in harsh conditions, are achieved by integrating a variety of sensors and connecting them over GSM sensor using ESP8266 modules. In addition, landslip detection is made more efficient and accurate with the use of AI approaches, which allows for the implementation of mitigation plans and early warnings. Ultimately, this integrated method offers a potential answer for early warning systems and landslide monitoring, which can help preserve lives and infrastructure in places prone to landslides across the globe. Communities at risk of landslides and other natural catastrophes can be made even more resilient through ongoing innovation and multidisciplinary cooperation

REFERENCES

1. Anh, G. Q., Duong, T. B., Solanki, V. K., & Tran, D. T. (2023). Landslide Monitoring System Using an IoT Wireless Sensor Network. In *Modern Approaches in IoT and Machine Learning for Cyber Security: Latest Trends in AI* (pp. 219-235). Cham: Springer International Publishing.
2. Bressani, L. A., Simões, E. B., Ogura, A. T., Boesing, I. J., Hanauer, D. S., UpSensor, N. H., & Balbinot, G. (2018). A monitoring system for landslides and geotechnical works using statistical and artificial intelligence models. In *Proceedings of 10th International Symposium on Field Measurements in Geomechanics*, Rio de Janeiro, Brazil.
3. Hakim, W. M., Aizat Faiz Ramli, Hafiz Basarudin, Mohd Azlan Abu, and Izanoordina Ahmad. "WSN and IoT based landslide monitoring system." *Test Engineering and Management* 83 (2020): 10926-10932.
4. Harsa, Hastuadi, Malinda Hidayat Anistia, Adi Mulsandi, Bambang Suprihadi, Roni Kurniawan, Muhammad Najib Habibie, Thahir Daniel Hutapea et al. "Machine learning and artificial intelligence models development in rainfall-induced landslide prediction." *IAES International Journal of Artificial Intelligence* 12, no. 1 (2023): 262.
5. Karunarathne, Sivanarayani M., Matthew Dray, Lyudmil Popov, Matthew Butler, Catherine Pennington, and Constantinos Marios Angelopoulos. "A technological framework for data-driven IoT systems: Application on landslide monitoring." *Computer Communications* 154 (2020): 298-312.
6. Khoa, V. V., & Takayama, S. (2018). Wireless sensor network in landslide monitoring system with remote data management. *Measurement*, 118, 214–229. doi: 10.1016/j.measurement.2018.01.002
7. Kumar, S., Duttagupta, S., Rangan, V. P., & Ramesh, M. V. (2019). Reliable network connectivity in wireless sensor networks for remote monitoring of landslides. *Wireless Networks*. doi:10.1007/s11276-019-02059-7

8. Ofli, F., Qazi, U., Imran, M., Roch, J., Pennington, C., Banks, V., & Bossu, R. (2022, July). A real-time system for detecting landslide reports on social media using artificial intelligence. In International Conference on Web Engineering (pp. 49-65). Cham: Springer International Publishing.
9. Saha, Anik, and Sunil Saha. "Integrating the artificial intelligence and hybrid machine learning algorithms for improving the accuracy of spatial prediction of landslide hazards in Kurseong Himalayan Region." *Artificial Intelligence in Geosciences* 3 (2022): 14-27.
10. Shutimarrungson, N., & Wuttidittachotti, P. (2019). Realistic propagation effects on wireless sensor networks for landslide management. *EURASIP Journal on Wireless Communications and Networking*, 2019(1), 1-14.